

DAY 02

WEEK LECTURE COURSE - DAY 02

클라이언트 컴포넌트와 LocalStorage 연동 Todo 앱

React Client Interaction 및 Hydration 에러 해결

Target: 대학 1학년 및 취업훈련생 입문 과정

Framework: Next.js 14/15 + Tailwind CSS

Date: 2026 Curriculum Edition

1. 목차 (Table of Contents)

- 학습 목표
- 도입: 서버 컴포넌트 vs 클라이언트 컴포넌트
- 개념 개론: React Hook (`useState` , `useEffect`)과 클라이언트 렌더링
- 핵심 기술: 수분 공급(Hydration)과 LocalStorage의 함정
- 실습: Mini-Project 2 - 인터랙티브 Todo 앱 (`/todo`)
- 실제 사례 연구: 쇼핑몰 장바구니와 로컬 저장소
- 결론 및 활용
- 강사 첨언 가이드 (Instructor Notes)

2. 학습 목표 (Learning Objectives)

1. Next.js에서 ****서버 컴포넌트(RSC)****와 ****클라이언트 컴포넌트(RCC)****의 차이를 이해하고 적절한 전환 시점을 파악한다.
2. `"use client"` 지시어의 쓰임새와 클라이언트 측 자바스크립트 실행 과정을 이해한다.
3. React Hook인 `useState` 와 `useEffect` 를 사용해 사용자 인터랙티브 데이터를 상태로 관리한다.
4. Next.js의 고유 현상인 ****수분 공급 오류(Hydration Error)****를 이해하고, 클라이언트 데이터 로딩 시 이를 극복하는 코드를 구현할 수 있다.

3. 도입: 서버 컴포넌트 vs 클라이언트 컴포넌트

Next.js는 기본적으로 모든 컴포넌트를 **서버 컴포넌트**로 간주합니다. 즉, 서버에서 미리 HTML을 다 그려서 전달하므로, 브라우저가 화면을 띄울 때 계산(연산)할 필요가 전혀 없게 만듭니다.

💡 직관적인 비유로 이해하는 서버/클라이언트 컴포넌트

- **서버 컴포넌트 (Server Component) - "완제품 배달 음식"**
 - 고객이 족발(HTML)을 주문하면, 가게 주방(서버)에서 완전히 다 삶아서 썰어 포장한 상태로 배달됩니다. 고객은 포장만 뜯어서 먹으면 됩니다. 가스 불을 켜거나 칼질을 할 필요가 없습니다. (정적 텍스트, DB 조회 데이터 표현에 적합)
- **클라이언트 컴포넌트 (Client Component) - "밀키트(Mealkit) 요리"**

- 고객이 삼겹살 밀키트(Javascript가 포함된 HTML)를 받습니다. 고기를 굽기 위해서는 고객 테이블(브라우저)에서 가스레인지 불을 켜고(이벤트 리스너), 고기가 익는 상태(`useState`)를 보며 직접 뒤집어 구워야 합니다. (버튼 클릭, 타이핑 입력, 슬라이더 조작 등에 적합)

사용자의 마우스 클릭, 입력(Input), 타이핑 같은 실시간 상호작용이 필요한 영역에는 반드시 컴포넌트 상단에 `"use client"` 라고 적어 주어야 합니다.

4. 개념 개론: React Hook과 클라이언트 렌더링

웹 서비스에 "동적 기능"을 부여하기 위해 두 가지 핵심 React Hook을 학습합니다.

1. `useState` : 컴포넌트 내부에서 유지되고 변화하는 "상태(State)" 변수입니다. 값이 변경되면 React가 감지하여 화면을 자동으로 다시 그려(Re-rendering) 줍니다.
2. `useEffect` : 컴포넌트가 화면에 나타날 때(Mount), 또는 특정 상태가 변할 때 실행되는 "부수 효과(Side Effect)" 처리 함수입니다. 외부 API 호출이나 브라우저 저장소 접근 시 활용됩니다.

5. 핵심 기술: Hydration의 함정과 로컬 저장소

초보 개발자가 Next.js에서 LocalStorage를 사용할 때 100% 확률로 직면하는 에러가 있습니다:

```
ReferenceError: window is not defined 또는 Hydration failed because the initial UI does not match what was rendered on the server.
```

? 원인은 무엇인가요?

Next.js 서버는 HTML을 미리 그릴 때 `window` 나 `localStorage` 같은 브라우저 전용 내장 객체를 알지 못합니다. (서버에는 브라우저가 없기 때문입니다.) 따라서 서버가 그린 초기 화면(빈 리스트)과 브라우저가 LocalStorage를 읽어와서 그린 화면(기존 Todo 목록)이 불일치하게 되면서 경고가 발생합니다.

🔧 해결 솔루션

컴포넌트가 브라우저에 성공적으로 안착(Mount)한 이후에만 `window / localStorage` 에 접근하도록 `useEffect` 와 `isMounted` 상태 플래그를 조합하여 제어합니다.

6. 실습 - Mini-Project 2: 인터랙티브 Todo 앱

이 프로젝트는 완성도 높은 애니메이션 효과와 필터링(전체/할일/완료), 그리고 LocalStorage 백업을 지원하는 모던한 할 일 관리 앱입니다.

 **소스코드:** `app/todo/page.tsx`

이 코드를 작성하여 `app/todo/page.tsx` 에 저장합니다.

```

'use client';

import { useState, useEffect } from 'react';
import Link from 'next/link';

interface Todo {
  id: number;
  text: string;
  completed: boolean;
  category: 'work' | 'personal' | 'study';
}

export default function TodoPage() {
  const [todos, setTodos] = useState<Todo[]>([]);
  const [inputText, setInputText] = useState('');
  const [category, setCategory] = useState<'work' | 'personal' | 'study'>('personal');
  const [filter, setFilter] = useState<'all' | 'active' | 'completed'>('all');
  const [isMounted, setIsMounted] = useState(false); // Hydration mismatch 해결용

  // 1. 브라우저 마운트 완료 확인 및 LocalStorage 로드
  useEffect(() => {
    setIsMounted(true);
    const saved = localStorage.getItem('antigravity_todos');
    if (saved) {
      try {
        setTodos(JSON.parse(saved));
      } catch (e) {
        console.error('로컬스토리지 데이터를 파싱하는 데 실패했습니다.', e);
      }
    }
  }, []);

  // 2. Todos 상태 변화 시 LocalStorage 자동 백업
  useEffect(() => {
    if (isMounted) {
      localStorage.setItem('antigravity_todos', JSON.stringify(todos));
    }
  }, [todos, isMounted]);

  // Todo 추가 핸들러
  const handleAddTodo = (e: React.FormEvent) => {
    e.preventDefault();
    if (!inputText.trim()) return;
  }

```

```

const newTodo: Todo = {
  id: Date.now(),
  text: inputText.trim(),
  completed: false,
  category: category,
};

setTodos([newTodo, ...todos]);
setInputText('');
};

// 완료 상태 토글
const handleToggleComplete = (id: number) => {
  setTodos(todos.map(todo =>
    todo.id === id ? { ...todo, completed: !todo.completed } : todo
  ));
};

// Todo 삭제
const handleDeleteTodo = (id: number) => {
  setTodos(todos.filter(todo => todo.id !== id));
};

// 필터링 적용된 목록 반환
const filteredTodos = todos.filter(todo => {
  if (filter === 'active') return !todo.completed;
  if (filter === 'completed') return todo.completed;
  return true;
});

// 카테고리 엠블럼 매핑 함수
const getCategoryStyles = (cat: string) => {
  switch (cat) {
    case 'work': return 'bg-blue-950 text-blue-400 border border-blue-800/30';
    case 'study': return 'bg-purple-950 text-purple-400 border border-purple-800/30';
    default: return 'bg-amber-950 text-amber-400 border border-amber-800/30';
  }
};

// Hydration 오류 방지를 위한 마운트 전 예외 렌더링
if (!isMounted) {
  return (
    <div className="min-h-screen bg-slate-950 text-slate-100 flex items-center justify-center">

```

```

    <p className="text-slate-500 animate-pulse">로딩 중...</p>
  </div>
);
}

return (
  <div className="min-h-screen bg-slate-950 text-slate-100 selection:bg-teal-500">
    <div className="max-w-2xl mx-auto space-y-8">

      {/* 상단 헤더 영역 */}
      <div className="flex justify-between items-center">
        <div className="space-y-1">
          <Link href="/" className="text-xs text-teal-400 hover:underline font-s">
            &larr; 홈으로 돌아가기
          </Link>
          <h1 className="text-3xl font-extrabold tracking-tight bg-gradient-to-r">
            Interactive To-Do
          </h1>
        </div>
        <div className="text-xs bg-slate-900 border border-slate-800 px-3 py-1.5">
          남은 할 일: <span className="text-teal-400 font-bold">{todos.filter(t
        </div>
      </div>

      {/* 할 일 입력 폼 */}
      <form onSubmit={handleAddTodo} className="bg-slate-900 border border-slate
        <div className="flex flex-col sm:flex-row gap-3">
          <input
            type="text"
            value={inputText}
            onChange={(e) => setInputText(e.target.value)}
            placeholder="오늘 해야 할 일을 기록해 보세요..."
            className="flex-grow bg-slate-950 border border-slate-800 rounded-xl
          />
          <button
            type="submit"
            className="px-6 py-3 bg-teal-500 text-slate-950 font-bold rounded-xl
          >
            할 일 추가
          </button>
        </div>

      {/* 카테고리 태그 선택기 */}
      <div className="flex items-center space-x-3 pt-2">

```

```

    <span className="text-xs font-semibold text-slate-500">카테고리:</span>
    {(['personal', 'work', 'study'] as const).map((cat) => (
      <button
        key={cat}
        type="button"
        onClick={() => setCategory(cat)}
        className={`px-3 py-1 text-xs rounded-full font-medium border transition-colors
          category === cat
            ? 'bg-teal-500 text-slate-950 border-teal-400 font-semibold'
            : 'bg-slate-950 text-slate-400 border-slate-800 hover:text-slate-500'
          }`}
      >
        {cat === 'personal' ? '개인' : cat === 'work' ? '업무' : '학습'}
      </button>
    ))}
  </div>
</form>

{/* 필터링 탭 */}
<div className="flex border-b border-slate-800 text-sm">
  {(['all', 'active', 'completed'] as const).map((t) => (
    <button
      key={t}
      onClick={() => setFilter(t)}
      className={`pb-3 px-4 font-semibold transition-all relative ${
        filter === t ? 'text-teal-400' : 'text-slate-400 hover:text-slate-500'
      }`}
    >
      {t === 'all' ? '전체 보기' : t === 'active' ? '진행 중' : '완료됨'}
      {filter === t && (
        <div className="absolute bottom-0 left-0 right-0 h-[2px] bg-teal-400" />
      )}
    </button>
  ))}
</div>

{/* 할 일 목록 */}
<div className="space-y-3">
  {filteredTodos.length === 0 ? (
    <div className="text-center py-16 bg-slate-900/30 border border-slate-800">
      <span className="text-3xl">🍵</span>
      <p className="text-slate-500 text-sm mt-3">등록된 할 일이 없습니다.</p>
    </div>
  ) : (

```

```

filteredTodos.map((todo) => (
  <div
    key={todo.id}
    className={`flex justify-between items-center p-4 bg-slate-900 border
      todo.completed ? 'border-slate-900/60 opacity-60' : 'border-slate-900/60 opacity-60'
    }`
  >
    <div className="flex items-center space-x-4 flex-grow min-w-0">
      {/* 체크박스 커스텀 UI */}
      <button
        onClick={() => handleToggleComplete(todo.id)}
        className={`w-5 h-5 rounded-md flex items-center justify-center
          todo.completed
            ? 'bg-teal-500 border-teal-500 text-slate-950'
            : 'border-slate-700 hover:border-teal-500'
          }`
      >
        <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 16 16">
          <path strokeLinecap="round" strokeLinejoin="round" d="M4.5 4.5 9 9" />
        </svg>
      </button>

      {/* 텍스트 내용 */}
      <span className={`text-sm select-none truncate pr-2 ${todo.completed ? 'text-slate-950' : 'text-slate-500'}`>
        {todo.text}
      </span>

      {/* 카테고리 태그 */}
      <span className={`px-2 py-0.5 rounded text-[10px] uppercase font-medium ${todo.category.toLowerCase().replace(' ', '-')}>
        {todo.category}
      </span>
    </div>

    {/* 삭제 버튼 */}
    <button
      onClick={() => handleDeleteTodo(todo.id)}
      className="p-1 text-slate-500 hover:text-red-400 transition-colors"
      aria-label="할 일 삭제"
    >
      <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 16 16">
        <path strokeLinecap="round" strokeLinejoin="round" d="M14.5 4.5 14.5 9 11 9 11 4.5 14.5 4.5" />
      </svg>
    </button>
  >
)
)

```

```

        </button>
      </div>
    ))
  })
</div>

</div>
</div>
);
}

```

7. 실제 사례 연구: 쇼핑몰 장바구니와 로컬 저장소

쿠팡이나 마켓컬리 같은 쇼핑몰의 장바구니 기능을 생각해 봅시다.

- **예시:** 로그인 전 장바구니에 아이템을 담고 새로고침해도 상품이 유지되는 기능.
- **원리:** 유저가 로그인하기 전에는 로컬 저장소(LocalStorage)에 선택된 상품 번호 리스트를 백업해 두었다가, 로그인 완료 시 서버 DB로 병합해 주는 하이브리드 아키텍처를 많이 사용합니다. 우리가 실습한 Todo 앱의 로컬스토리지 동기화 원리와 동일합니다.

8. 결론 및 활용

- **요약:** "use client" 는 단순 HTML 정적 페이지인 Next.js에 자바스크립트의 숨결을 불어넣는 도구입니다. `useState` 와 `useEffect` 를 사용하면 실시간 상태 업데이트와 반영구 데이터 영속화를 구축할 수 있습니다.
- **숙제:** Todo 앱에 '중요도(별표)'를 체크하는 아이콘 버튼과 중요도 필터를 추가하는 기능을 코드에 녹여 보세요.

9. 강사 첨언 가이드 (Instructor Notes)

💡 TIP

수업 진행 가이드

- **00m ~ 30m:** React Server/Client Component를 '밀키트와 배달음식' 비유로 가볍게 설명하고 칠판에 영역을 매핑합니다.
- **30m ~ 45m:** LocalStorage 접근 시 Next.js에서 빌드 타임에 에러를 뿜는 원인을 터미널 창을 직접 보여주며 학습시킵니다.
- **45m ~ 90m:** `app/todo/page.tsx` 소스코드 실습을 진행하며 마운트 가드 코드(`isMounted`)의 기술적 필요성을 명료하게 설명하세요.

🔥 CAUTION

자주 발생하는 에러 및 대처법

1. **Hydration Mismatch 경고:** 마운트 가드 코드(`if (!isMounted) return ...`)를 빼먹으면 브라우저 콘솔 창이 온통 붉은 경고로 덮입니다. 기능은 작동하더라도 배포 시 SEO 최적화와 UX 품질에 치명적임을 설명해 주세요.
2. **JSON.parse 에러:** 로컬스토리지에 기존의 텍스트 포맷이나 오염된 문자열이 저장되어 있는 경우 `JSON.parse` 도중 앱이 폭발합니다. 반드시 `try-catch` 구문으로 예외처리를 하도록 가이드해야 완성도 높은 코딩 습관이 길러집니다.